Accesibilidad en interfaces

Reglas

- Solo se toca la carpeta `public/`. No toques `pages/`, `data/`, `next.config.js` ni `package.json`.
- Usa **Chrome o Edge**. Firefox y Safari no tienen reconocimiento de voz.
- Abre siempre `http://localhost:3000`. La cámara y el micrófono solo funcionan ahí o en HTTPS.
- Los ejercicios de mano necesitan Internet, porque descargan MediaPipe.
- Comenta tu código, línea a línea, igual que los ejercicios resueltos.

Cómo arrancar

1. Instala Node.js 18 o superior desde https://nodejs.org.
2. Descomprime el ZIP.
3. En Windows, haz doble clic en `start.bat`. En Linux o Mac, ejecuta `chmod +x start.sh` y luego `./start.sh`.
4. La primera vez tarda un poco, porque instala las librerías.
5. Se abre el navegador en `http://localhost:3000`. Si no se abre, escríbelo tú.
6. Entra en cada ejercicio desde la página de inicio.
7. Cuando cambies un archivo, guarda y pulsa F5 en el navegador.

Cómo estudiar

1. Lee el resuelto del mismo tipo (E1 con R1 y R2, E3 con R3, E4 con R4, E5 con R5, E6 con R6, E7 con R7).
2. Pruébalo y entiende cada línea.
3. Abre tu evaluable y busca `TODO` (en VS Code: `Ctrl+F`).
4. Haz un `TODO` cada vez, guarda y prueba.

Si algo falla

- **No funciona nada:** abre las herramientas de desarrollador con `F12`, entra en "Consola" y lee el error en rojo.
- **No pide permiso de micrófono o cámara:** pulsa el candado junto a la dirección y permite el acceso.
- **"Usa Chrome o Edge":** cambia de navegador.

- **La mano no se detecta:** comprueba que tienes Internet y buena luz.

Qué entregar

- La carpeta `public/` completa, comprimida en un ZIP.
- Tu nombre en un comentario al inicio de cada archivo que modifiques.

E1. Dictar un cliente

Criterio: RA2 c (reconocimiento de voz). **Peso:** 1,5 puntos.
Archivo: `public/evaluables/e1-voz-formulario.html`

Objetivo: rellenar un formulario con la voz y guardarlo en el servidor.

Cómo funciona. Pulsas "Hablar" y dices una frase. Cada pulsación escucha **una sola frase**, así que repites el botón para cada campo.

Dices	Debe pasar
"nombre Ana López"	`Ana López` aparece en el cuadro Nombre
"ciudad Madrid"	`madrid` aparece en el cuadro Ciudad
"email ana arroba mail punto es"	`ana@mail.es` aparece en el cuadro Email
"guardar"	Sale el mensaje "Guardado"

PROMETEO by thePower

Pasos:

1. **TODO 1.** Dentro de `oido.onresult`, comprueba si `frase` empieza por `"nombre "`. Usa `frase.startsWith('nombre ')`. Si es así, pon el resto de la frase en el input `#name`. El resto se obtiene con `frase.slice(7)`, porque `"nombre "` tiene 7 letras.
2. **TODO 2.** Haz lo mismo con `"ciudad "` (7 letras, input `#city`) y `"email "` (6 letras, input `#email`).
3. En el email, el navegador escribe las palabras tal cual. Sustituye `" arroba "` por `"@"` y `" punto "` por `"."`, y quita los espacios restantes. Usa `.replace(/ arroba /g, '@')`.
4. **TODO 3.** Si la frase es exactamente `"guardar"`, crea un objeto `{ name, city, email }` con los valores de los inputs. Envíalo con `fetch('/api/customers', {...})`. Usa `method: 'POST'`, la cabecera `'Content-Type': 'application/json'` y `body: JSON.stringify(datos)`.
5. Escribe en `#msg` "Guardado" si `r.ok` es verdadero y un mensaje de error si no.

Cómo comprobarlo:

- Abre `http://localhost:3000/api/customers` en otra pestaña. Debe aparecer el cliente nuevo al final de la lista.

Errores típicos:

- Olvidar `.toLowerCase()`. Ya viene hecho, pero respeta las minúsculas al comparar.
- Olvidar el espacio al final de `"nombre "`.
- Olvidar `async` y `await` al llamar a `fetch`.

PROMETEO
by thePower

E2. Filtrar productos con la voz

Criterio: RA2 b y c. **Peso:** 1,5 puntos.
Archivo: `public/evaluables/e2-voz-productos.html`

Objetivo: mostrar una tabla de productos y filtrarla por voz.

Dices	Debe pasar
(al cargar)	Tabla con todos los productos
"stock bajo"	Solo productos con stock menor que 5
"todos"	Vuelven todos los productos
"buscar teclado"	Solo productos cuyo nombre contiene "teclado"

Pasos:

1. **TODO 1.** En `pintar(lista)`, construye una fila `<tr>` por producto. Cada fila lleva tres celdas `<td>`: nombre, precio y stock.
2. Usa `lista.map(...)` para crear las filas y `.join('')` para unirlas. Añade antes una fila de cabecera con `<th>`.
3. Asigna el resultado a `document.getElementById('tabla').innerHTML`.
4. **TODO 2.** Si la frase incluye `"stock bajo"`, llama a `pintar(productos.filter((p) => p.stock < 5))`.
5. **TODO 3.** Si incluye `"todos"`, llama a `pintar(productos)`.
6. **TODO 4.** Si empieza por `"buscar "`, saca la palabra con `frase.slice(7)`. Filtra los productos con `p.name.toLowerCase().includes(palabra)`.

E3. Pasar productos moviendo el brazo

Criterio: RA2 d (movimiento del cuerpo). **Peso:** 1,5 puntos.
Archivo: `public/evaluables/e3-movimiento-productos.html`

Objetivo: cambiar de producto al mover el brazo, y mostrarlo con el componente `<erp-tarjeta>`.

Cómo funciona. El programa compara dos fotos seguidas de la cámara. Si muchos píxeles han cambiado, hay movimiento. Mira si ocurrió en la mitad izquierda o en la derecha de la imagen.

Haces	Debe pasar
Mover el brazo a la derecha	Producto siguiente
Mover el brazo a la izquierda	Producto anterior

Pasos:

1. **TODO 1.** En `mostrar()`, guarda en una variable el producto `productos[actual]`. Después pon `document.getElementById('tarjeta').titulo = p.name` y `.valor = p.stock`.
2. **TODO 2.** Dentro del `if (anterior)`, recorre `foto` con un `for` de 4 en 4 (cada píxel ocupa 4 números: R, G, B y A).
3. Calcula el brillo con `foto[i] + foto[i+1] + foto[i+2]` y el de la foto anterior de la misma manera.
4. Si la diferencia absoluta supera 90, hubo cambio. Calcula la columna con `(i / 4) % 160`. Si es menor que 80, suma a `izq`. Si no, suma a `der`.
5. **TODO 3.** Si ha pasado el tiempo de bloqueo (`performance.now() > bloqueo`) y `izq` o `der` supera 600, cambia el producto. Si `der > izq`, avanza; si no, retrocede. Después llama a `mostrar()`.
6. Pon `bloqueo = performance.now() + 1000`. Así evitas cambiar diez veces por un solo gesto.

Consejos para probar:

- Buena iluminación y fondo quieto.
- Mueve el brazo entero y con ganas, no solo los dedos.
- Si cambia solo, sube el 600. Si no reacciona, bájalo.

Errores típicos:

- Olvidar que al retroceder desde 0 hay que volver al último producto. Usa `(actual + productos.length - 1) % productos.length`.
- Olvidar `anterior = foto.slice()` al final.

E4. Cambiar el estado de un pedido con la mano

Criterio: RA2 e (detección de partes del cuerpo). **Peso:** 1,5 puntos.
Archivo: `public/evaluables/e4-mano-pedidos.html`

Objetivo: usar el dedo índice como puntero y la **pinza** (pulgar e índice juntos) como clic.

Pasos:

1. **TODO 1.** Con `m = r.landmarks[0]`, el índice es `m[8]` y el pulgar es `m[4]`. Calcula `caja = zona.getBoundingClientRect()`.
2. Calcula `x = (1 - m[8].x) * caja.width` e `y = m[8].y * caja.height`. El `1 -` es porque el vídeo está en espejo.
3. Asigna `punto.style.left = x + 'px'` y `punto.style.top = y + 'px'`.
4. **TODO 2.** La distancia entre dedos es `Math.hypot(m[8].x - m[4].x, m[8].y - m[4].y)`. Si es menor que 0.05, hay pinza. Añade o quita la clase `pinza` con `punto.classList.toggle('pinza', pinza)`.

5. Pulsa solo en el instante en que juntas los dedos, es decir, si `pinza && !pinzaAntes`. Obtén el elemento con `document.elementFromPoint(caja.left + x, caja.top + y)`. Si es un `BUTTON` dentro de la zona, llama a `el.click()`.
6. Al final guarda `pinzaAntes = pinza`.
7. **TODO 3.** En el evento `click`, si hay `estado`, haz `fetch('/api/orders?id=1', {...})` con `method: 'PUT'`, JSON y `body: JSON.stringify({ status: estado })`. Muestra el resultado en `#msg`.

Cómo comprobarlo:

- Abre `http://localhost:3000/api/orders`. El pedido 1 debe tener el estado que pulsaste.

Errores típicos:

- Olvidar el espejo y que el punto se mueva al revés.
- Pulsar sin `pinzaAntes`, que provoca cientos de clics seguidos.

E5. Crear el componente `<erp-insignia>`

Criterio: RA3 b, c, d y h. **Peso:** 1,5 puntos.
Archivos: `public/componentes/erp-insignia.js` y `public/evaluables/e5-componente-insignia.html`

Objetivo: crear tu propia etiqueta HTML que muestra un estado con color.

Estado	Fondo	Texto
pending	`#fff0cc`	`#8a5a00`
paid	`#d6f5e0`	`#17753a`
shipped	`#dbe7ff`	`#1d4ed8`
cancelled	`#fbdada`	`#a12222`

Pasos en `erp-insignia.js`:

1. **TODO 1.** Añade `static get observedAttributes() { return ['estado']; }` y `attributeChangedCallback() { this.pintar(); }`.
2. **TODO 2.** Crea el `get estado()`. Si el atributo no está en la lista `['pending','paid','shipped','cancelled']`, devuelve `'pending'`. Crea también `set estado(v)`.
3. **TODO 3.** En `connectedCallback`, añade un `click`. Calcula el siguiente estado de la lista (al llegar al último vuelve al primero) y guárdalo en `this.estado`. Después lanza `new CustomEvent('cambio', { detail: { estado }, bubbles: true, composed: true })`.
4. **TODO 4.** En `pintar()`, escribe en `this.shadowRoot.innerHTML` una etiqueta `<style>` y un `<span>` con el estado. Usa los colores de la tabla.

Pasos en la página:

1. Carga el componente con `await import('/componentes/erp-insignia.js')`.
2. Con `document.addEventListener('cambio', ...)`, añade un `<li>` a `#log` con el nuevo estado.

Cómo comprobarlo:

- La primera insignia (sin atributo) muestra `pending`. Las otras muestran `paid` y `shipped`.
- Cada clic cambia el estado y añade una línea al registro.

E6. Pruebas, documentación y paquete

Criterio: RA3 e, f y g. **Peso:** 1 punto.
Archivos: `public/evaluables/e6-pruebas-paquete.html` y `public/componentes/README.md`

Parte 1: cuatro pruebas unitarias. Mira R6 como modelo. Cada una usa `test('nombre', () => {...}, salida)` y `igual(real, esperado)`.

1. El estado por defecto es `pending`.
2. Lee bien el estado del atributo (`estado="paid"` da `paid`).
3. Un estado inválido (`"xyz"`) devuelve `pending`.
4. Al hacer clic se lanza el evento `cambio` y pasa al estado siguiente.

Crea el componente con `document.createElement('erp-insignia')` y añádelo a la zona de pruebas con `zona.appendChild(...)`. Los cuatro resultados deben salir en verde.

Parte 2: documentación. En `public/componentes/README.md`, bajo `## erp-insignia`, escribe:

- Una tabla de propiedades (nombre, valor por defecto, descripción).
- El evento `cambio` y los datos que lleva.
- Un ejemplo de uso en HTML.

Parte 3: paquete.

1. En la terminal, dentro del proyecto, ejecuta `npm run empaquetar`.
2. Abre `public/dist/erp-componentes.js` y comprueba que contiene tus dos componentes.

E7. Proyecto integrado

Criterio: RA2 b, c, d, e y f, y RA3 h. **Peso:** 1,5 puntos.
Archivo: `public/evaluables/e7-proyecto-integrado.html`

Objetivo: juntar voz, mano, realidad aumentada y componentes en una sola página de pedidos. Necesitas haber terminado E5.

Parte	Debe pasar
Lista	Cada pedido muestra cliente, total e `<erp-insignia>` con su estado
Voz	"pagados" muestra solo los pagados y "todos" muestra todos
Mano	Una tarjeta flota sobre la mano con el pedido actual. La pinza pasa al siguiente

Pasos:

1. **TODO 1.** En `pintarLista()`, escribe en `#lista` una línea por pedido de `visibles`. Cada línea lleva `p.customer`, `p.total` y `<erp-insignia estado="${p.status}">`.
2. **TODO 2.** Crea el reconocimiento de voz como en E2. Si la frase incluye "pagado", pon `visibles = pedidos.filter((p) => p.status === 'paid')`. Si incluye "todos", pon `visibles = pedidos`. Reinicia `i = 0` y llama a `pintarLista()`.
3. **TODO 3.** Crea el bucle de la mano como en R7. Si no hay mano, oculta `#ar`. Si la hay, colócala sobre `m[9]` (centro de la mano) teniendo en cuenta el espejo.
4. Escribe en la tarjeta cliente, total y estado de `visibles[i]`. Con la pinza, pasa `i` al siguiente, sin salirte de la lista.
5. Pon un botón o aviso para pedir permiso de cámara y micrófono.

Errores típicos:

- Que `i` se salga de la lista tras filtrar. Usa `i % visibles.length`.
- Olvidar repintar la lista tras la orden de voz.